

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is an architectural pattern that optimizes the output of a Large Language Model (LLM) by referencing a specific, authoritative knowledge base outside of its initial training data.

Why it is Needed

Standard LLMs suffer from "knowledge cutoff" and "hallucinations." In a customer support context, an LLM might invent a return policy. RAG ensures the model's response is grounded in the company's actual PDF documentation.

Use Case Overview

The **Customer Support Assistant** automates responses to policy-related questions while identifying high-risk or complex queries that require a human expert, ensuring both efficiency and legal compliance.

2. System Architecture Explanation

The system follows a modular architecture where the **Data Ingestion** and **Query Orchestration** are decoupled.

- **Ingestion:** Processes the PDF into a searchable vector format.
- **Interaction:** A graph-based state machine (LangGraph) manages the flow from user input to final response.

3. Design Decisions

- **Chunk Size (1000 characters):** Selected to ensure each chunk contains enough semantic "weight" to be useful (a full paragraph) without exceeding the model's context window when multiple chunks are retrieved.
- **Embedding Strategy:** Utilized all-MiniLM-L6-v2 via HuggingFace. This choice provides a 384-dimensional vector space, offering high accuracy for English text while running locally to minimize latency.
- **Retrieval Approach:** K-Nearest Neighbors (k=3) via ChromaDB. Retrieving the top 3 chunks provides a balance between sufficient context and noise reduction.
- **Prompt Design Logic:** Used a "System-Human" prompt structure. The System prompt strictly instructs the LLM: *"Use ONLY the provided context. If the answer is not there, admit you do not know."*

4. Workflow Explanation

LangGraph was utilized to move beyond simple linear chains into a stateful, agentic workflow.

- **Node Responsibilities:**
 - **intent_analyzer:** Pre-processes the query to determine the path.
 - **rag_engine:** Fetches context and generates AI responses.
 - **human_expert:** Pauses execution for manual input.
- **State Transitions:** The AgentState dictionary carries the query and response through the graph, ensuring data persistence across nodes.

5. Conditional Logic

- **Intent Detection:** The system performs a keyword-based and semantic check on the user's query.
- **Routing Decisions:** If "refund," "legal," or "sue" is detected, the router updates the state's next_action to human. Otherwise, it defaults to ai.

6. HITL Implementation

Human-in-the-Loop (HITL) serves as a fail-safe.

- **Role:** To handle queries where the AI might be unreliable or where company policy dictates a human touch (e.g., legal threats).
- **Benefits:** Reduces liability and increases customer trust.
- **Limitations:** Introduces a bottleneck (human response time) compared to instant AI answers.

7. Challenges & Trade-offs

- **Accuracy vs. Speed:** Using a local embedding model (MiniLM) is faster than calling a cloud API but slightly less nuanced than models like text-embedding-3-large.
- **Chunk Size vs. Context Quality:** Smaller chunks (500) provide more specific matches but lose the "surrounding story," whereas larger chunks (2000) might include irrelevant text that confuses the LLM.
- **Cost vs. Performance:** Groq (Llama 3.3) was chosen for its extreme speed and low cost, providing "GPT-4 level" reasoning without the high per-token price tag.

8. Testing Strategy

- **Approach:** Unit testing each node in isolation, followed by integration testing of the full graph.
- **Sample Query 1:** "What is the return policy?" → *Result: Correct RAG response.*
- **Sample Query 2:** "I will sue you!" → *Result: Successful HITL escalation.*

9. Future Enhancements

- **Multi-document Support:** Implementing a "Router" that picks the correct PDF based on department (e.g., Billing PDF vs. Technical PDF).
- **Feedback Loop:** Allowing users to rate the AI answer; low ratings automatically trigger a human review of the RAG chunks.
- **Memory Integration:** Using LangGraph's Checkpointer to allow the bot to remember a user across multiple days/sessions.